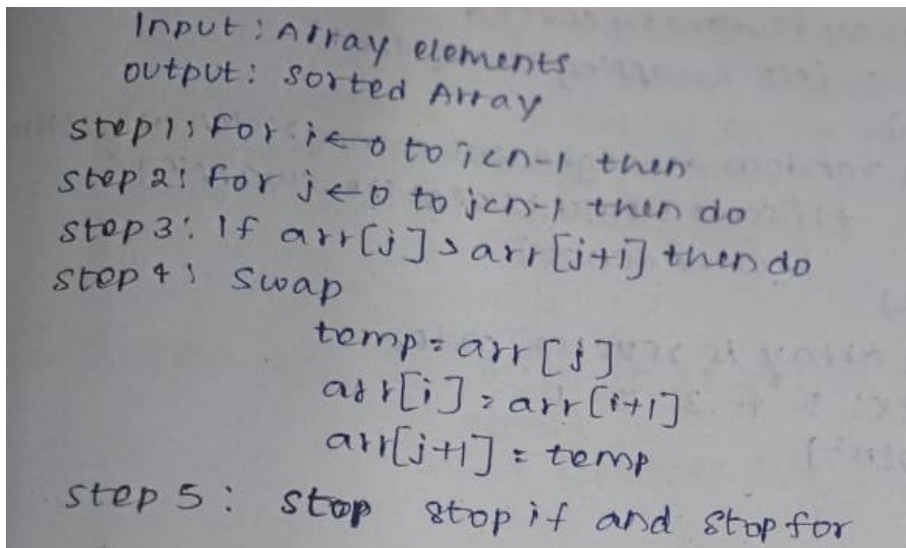


Practical - 1

Aim: To implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

1) Bubble Sort

Algorithm:



Input: Array elements
output: Sorted Array
step 1: For $i \leftarrow 0$ to $n-1$ then
step 2: For $j \leftarrow 0$ to $j \leftarrow n-1$ then do
step 3: If $arr[j] > arr[j+1]$ then do
step 4: Swap
 $temp = arr[j]$
 $arr[j] = arr[j+1]$
 $arr[j+1] = temp$
step 5: stop stop if and stop for

Program:

```
#include <iostream>
using namespace std;

class BubbleSort
{
public:
    int arr[100], n;

    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
}
```



```
void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void logic()
{
    int temp;

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

};

int main()
{
    BubbleSort bs;

    bs.arrayInput();

    cout << "\nBefore Sorting: ";
    bs.printArray();

    bs.logic();

    cout << "After Sorting: ";
    bs.printArray();

    return 0;
}
```

Output:

```
enter program name, for example C:\Users\vemal\OneDrive\Documents\DSA/a.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ bubble.cpp -o bubble.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\bubble.exe
Enter number of elements: 5
Enter the elements: 19 22 12 16 30

Before Sorting: 19 22 12 16 30
After Sorting: 12 16 19 22 30
PS C:\Users\vemal\OneDrive\Documents\DSA> |
```

Time Complexity:

```
for (int i = 0; i < n-1; i++) {
    for (int j = 0; j < n-1-i; j++) {
        if (arr[j] > arr[j+1]) {
            temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}
```

$T = (n-1)(n-1) \times 1 \times 1 \times 1$
 $T = (n-1)^2$
 $T = n^2 - 1 - 2n$
 $T \approx n^2 - n$
 $T(n) = O(n^2)$

Best case: If the array is already sorted
Ex: 1 2 3 4 5 (No swapping is required)
 $T(n) = O(n)$

Average case: For a random arrangement comparisons are still
 $(n-1) + (n-2) + \dots + 1$ (some elements require swapping & some don't)
 $T(n) = O(n^2)$

Worst case: If the array is reverse order
Ex: 5 4 3 2 1
 $T(n) = O(n^2)$

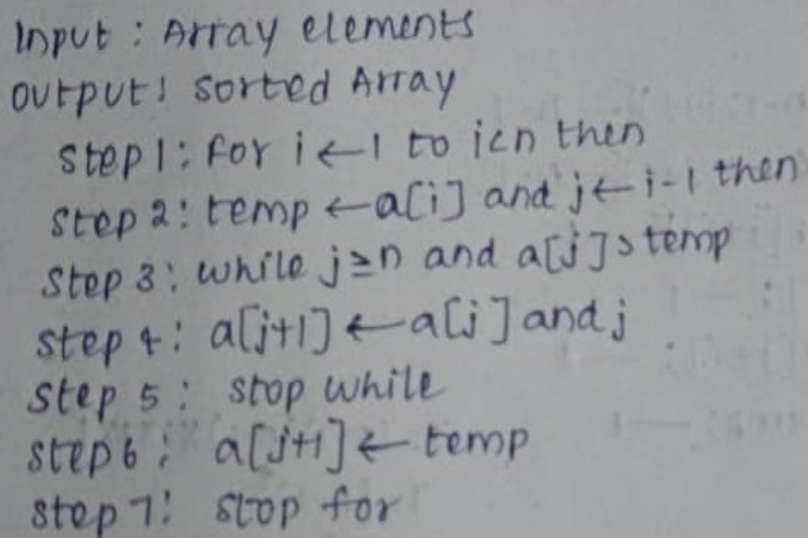
Input: Array elements

Bubble sort Conclusion :

Thus, the Bubble Sort algorithm was successfully implemented and studied. It sorts the elements by repeatedly comparing adjacent elements and swapping them when they are in the wrong order. It is simple and easy to understand, but it is less efficient for large datasets because its average and worst-case time complexity is $O(n^2)$.

2) Selection sort:

Algorithm:



Input : Array elements
Output : sorted Array
Step 1: for $i \leftarrow 1$ to $i \leftarrow n$ then
Step 2: $temp \leftarrow a[i]$ and $j \leftarrow i-1$ then
Step 3: while $j \geq n$ and $a[j] > temp$
Step 4: $a[j+1] \leftarrow a[j]$ and j
Step 5: stop while
Step 6: $a[j+1] \leftarrow temp$
Step 7: stop for

Program :

```
#include <iostream>
using namespace std;

class SelectionSort
{
public:
    int arr[100], n;

    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }

    void logic()
    {
        int i, j, min, temp;

        for (i = 0; i < n - 1; i++)
        {
            min = i;
```



```
        for (j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[min])
            {
                min = j;
            }
        }

        swap(arr[i], arr[min]);
    }
}

void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

    cout << endl;
}

};

int main()
{
    SelectionSort ss;

    ss.arrayInput();

    cout << "Before Sorting: ";
    ss.printArray();

    ss.logic();

    cout << "After Sorting: ";
    ss.printArray();

    return 0;
}
```

Output:

```
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ selection.cpp -o selection.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\selection.exe
Enter number of elements: 5
Enter the elements: 22 30 12 16 19
Before Sorting: 22 30 12 16 19
After Sorting: 12 16 19 22 30
```

Time complexity:

for (i=1; i<n; i++) { $\rightarrow n$
temp = a[j]; $\rightarrow 1$
j = i-1; $\rightarrow 1$
while (j > 0 && a[j] > temp) { $\rightarrow n$
a[j+1] = a[j]; $\rightarrow 1$
j--; $\rightarrow 1$
}
a[j+1] = temp; $\rightarrow 1$
}

$T = n \times n$
 $T(n) = n^2$
 $T(n) = O(n^2)$

Best case: The array is already sorted
Eg: 10 20 30 40 50 (Even though the array is sorted selection sort still checks every remaining element to find the minimum)
 $\Rightarrow (n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$
 $\therefore T(n) = O(n^2)$

Average case: Eg: 30 10 50 20 40 (The elements are in random order)
for n elements $\frac{n(n-1)}{2}$
 $\therefore T(n) = O(n^2)$

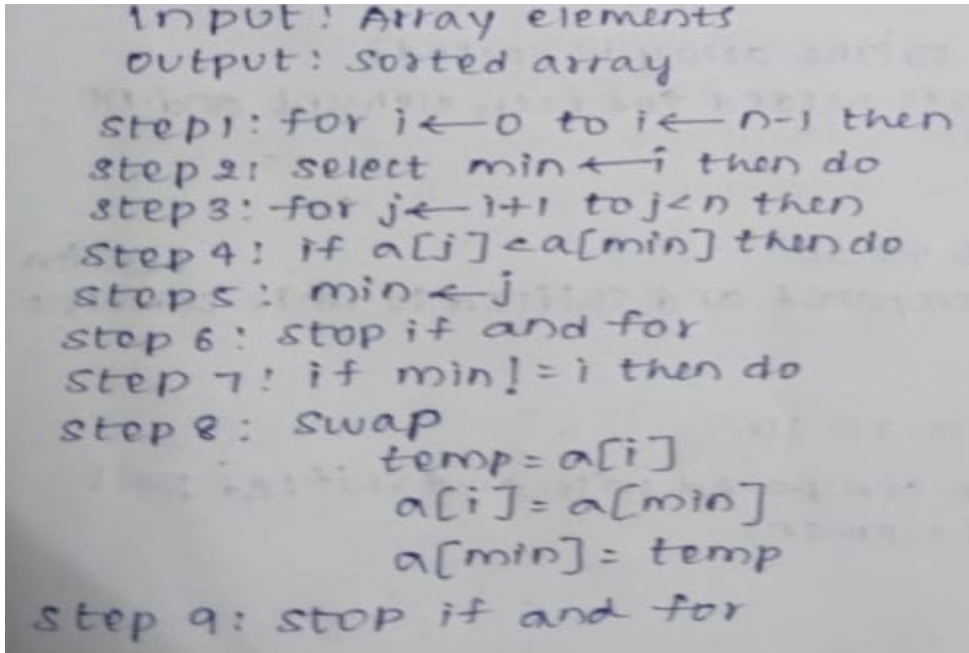
Worst case: Eg: 50 40 30 20 10 (The array is in reverse order)
The no. of comparisons remains: $\frac{n(n-1)}{2}$
 $\therefore T(n) = O(n^2)$

Selection sort Conclusion :

Thus, the Selection Sort algorithm was successfully implemented and studied. It sorts the elements by repeatedly finding the smallest element from the unsorted part and placing it in the correct position. It is simple and easy to understand, but its time complexity is **$O(n^2)$** in the best, average, and worst cases.

3) Insertion sort:

Algorithm:



Input: Array elements
Output: Sorted array

step 1: for $i \leftarrow 0$ to $i \leftarrow n-1$ then
step 2: select $\text{min} \leftarrow i$ then do
step 3: for $j \leftarrow i+1$ to $j < n$ then
step 4: if $a[j] < a[\text{min}]$ then do
step 5: $\text{min} \leftarrow j$
step 6: stop if and for
step 7: if $\text{min} \neq i$ then do
step 8: swap
 $\text{temp} = a[i]$
 $a[i] = a[\text{min}]$
 $a[\text{min}] = \text{temp}$
step 9: stop if and for

Program:

```
#include <iostream>
using namespace std;

class InsertionSort
{
public:
    int arr[100], n;

    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }

    void logic()
    {
        int i, j, key;
```



```
        for (i = 1; i < n; i++)
        {
            key = arr[i];

            while (j >= 0 && arr[j] > key)
            {
                arr[j + 1] = arr[j];
                j--;
            }

            arr[j + 1] = key;
        }
    }

    void printArray()
    {
        for (int i = 0; i < n; i++)
        {
            cout << arr[i] << " ";
        }

        cout << endl;
    }
};

int main()
{
    InsertionSort is;

    is.arrayInput();

    cout << "Before Sorting: ";
    is.printArray();

    is.logic();

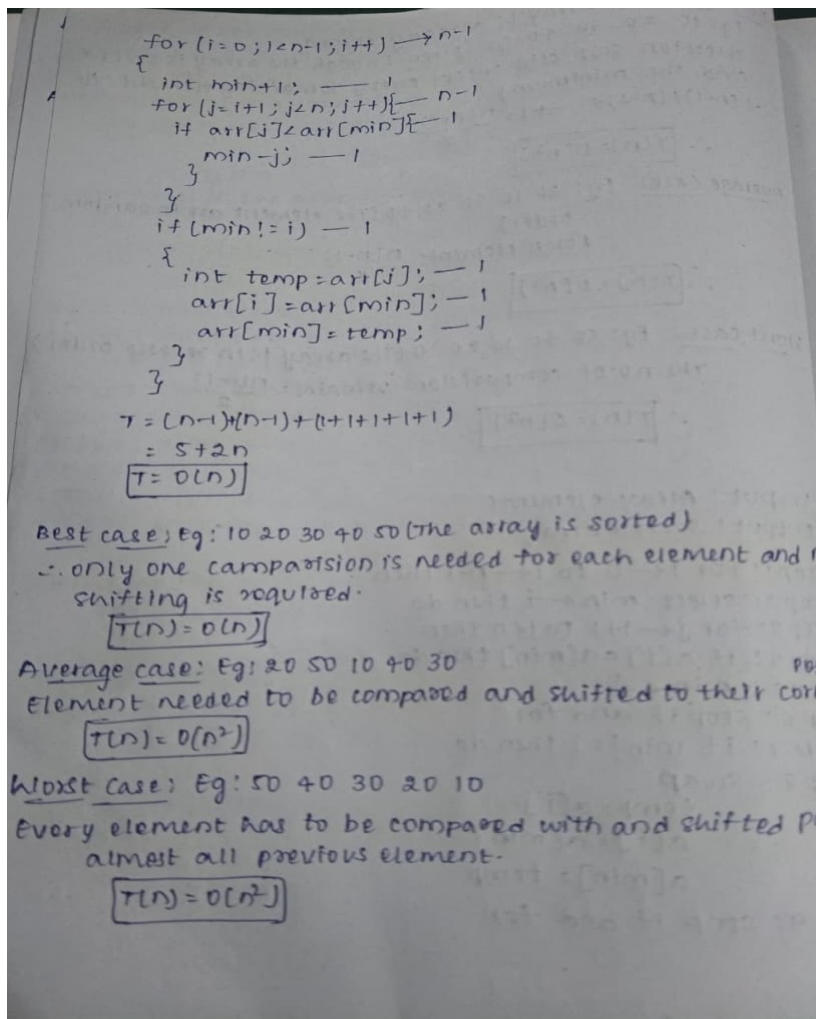
    cout << "After Sorting: ";
    is.printArray();

    return 0;
}
```

Output:

```
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ insertion.cpp -o insertion.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\insertion.exe
Enter number of elements: 5
Enter the elements: 22 12 16 19 30
Before Sorting: 22 12 16 19 30
After Sorting: 12 16 19 22 30
```

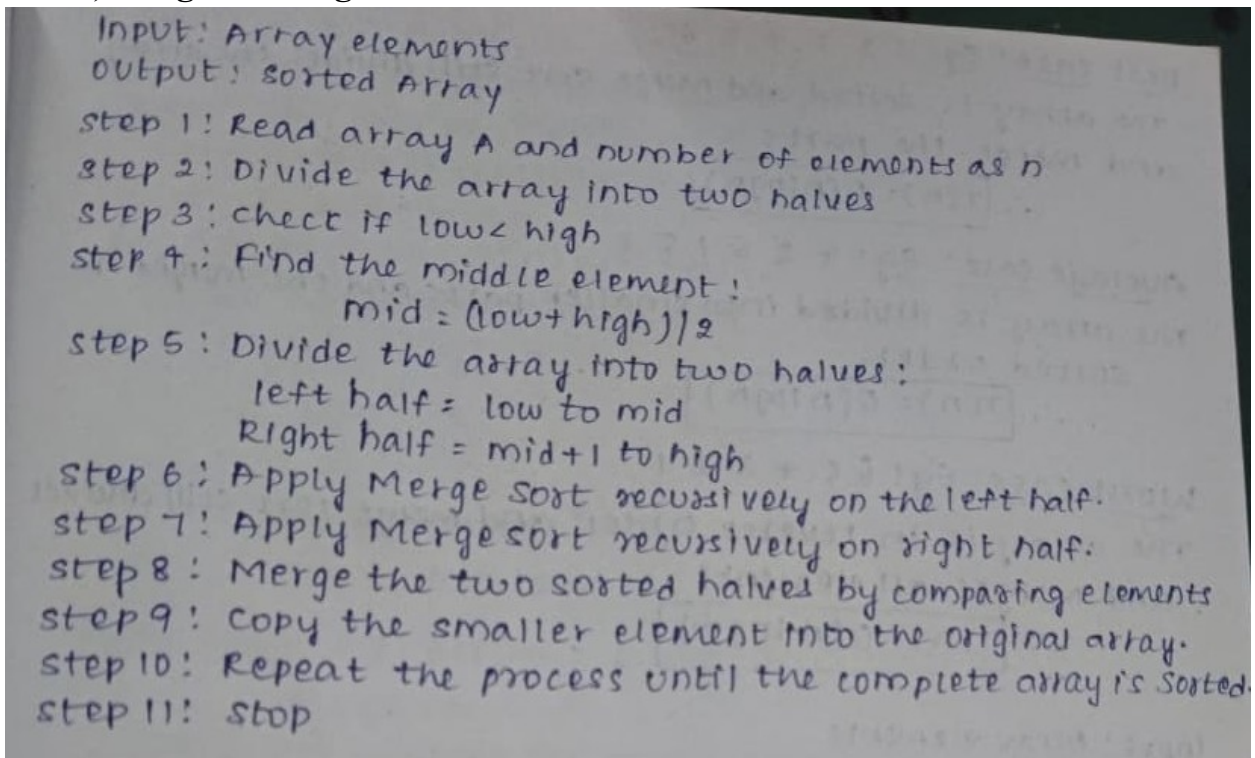
Time Complexity :



Insertion sort Conclusion :

Thus, the Insertion Sort algorithm was successfully implemented and studied. It sorts the elements by taking one element at a time and inserting it into its correct position in the sorted part of the array. It is simple and efficient for small or nearly sorted data. Its best-case time complexity is $O(n)$, while its average and worst-case time complexity is $O(n^2)$.

4) Merge Sort Algorithm :



Input: Array elements
Output: sorted Array

- step 1: Read array A and number of elements as n
- step 2: Divide the array into two halves
- step 3: check if $low < high$
- step 4: Find the middle element:
 $mid = (low + high) / 2$
- step 5: Divide the array into two halves:
left half = low to mid
Right half = mid+1 to high
- step 6: Apply Merge Sort recursively on the left half.
- step 7: Apply Merge sort recursively on right half.
- step 8: Merge the two sorted halves by comparing elements
- step 9: copy the smaller element into the original array.
- step 10: Repeat the process until the complete array is sorted.
- step 11: stop

Program :

```
#include <iostream>
using namespace std;

class MergeSort
{
public:
    int arr[100], n;

    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;

        cout << "Enter the elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
    }
}
```



```
void merge(int low, int mid, int high)
{
    int temp[100];

    int i = low;
    int j = mid + 1;
    int k = low;

    while (i <= mid && j <= high)
    {
        if (arr[i] < arr[j])
        {
            temp[k] = arr[i];
            i++;
        }
        else
        {
            temp[k] = arr[j];
            j++;
        }

        k++;
    }

    while (i <= mid)
    {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= high)
    {
        temp[k] = arr[j];
        j++;
        k++;
    }

    for (i = low; i <= high; i++)
    {
        arr[i] = temp[i];
    }
}
```

```
void logic(int low, int high)
{
```



```
    if (low < high)

    {
        int mid = (low + high) / 2;

        logic(low, mid);
        logic(mid + 1, high);
        merge(low, mid, high);
    }
}

void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

    cout << endl;
}

};

int main()
{
    MergeSort ms;

    ms.arrayInput();

    cout << "Before Sorting: ";
    ms.printArray();

    ms.logic(0, ms.n - 1);

    cout << "After Sorting: ";
    ms.printArray();

    return 0;
}
```

Output:

```
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ merge.cpp -o merge.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\merge.exe
Enter number of elements: 5
Enter the elements: 10 20 40 22 19
Before Sorting: 10 20 40 22 19
After Sorting: 10 19 20 22 40
```

Time Complexity :

$\text{Merge sort}(\text{arr}, \text{low}, \text{mid}); \rightarrow n/2$
 $\text{Merge sort}(\text{arr}, \text{mid}+1, \text{high}); \rightarrow n/2$
 $\text{Merge sort}(\text{arr}, \text{low}, \text{mid}, \text{high}); \rightarrow n$
 $T(n) = T(n/2) + T(n/2) + n$
 $T(n) = 2T(n/2) + n$
Master Method: $T(n) = aT(n/b) + f(n)$
 $a=2, b=2, f(n)=n, p=0, k=1$
 $n \log_b^a = n \log_2^2 = n$
 $f(n)=n$
 $n \log_b^c = n \therefore \log_b^a = k$
 case 2: $p = -1$
 $T(n) = O(n \log^{p+1} n)$
 $T(n) = O(n \log n)$

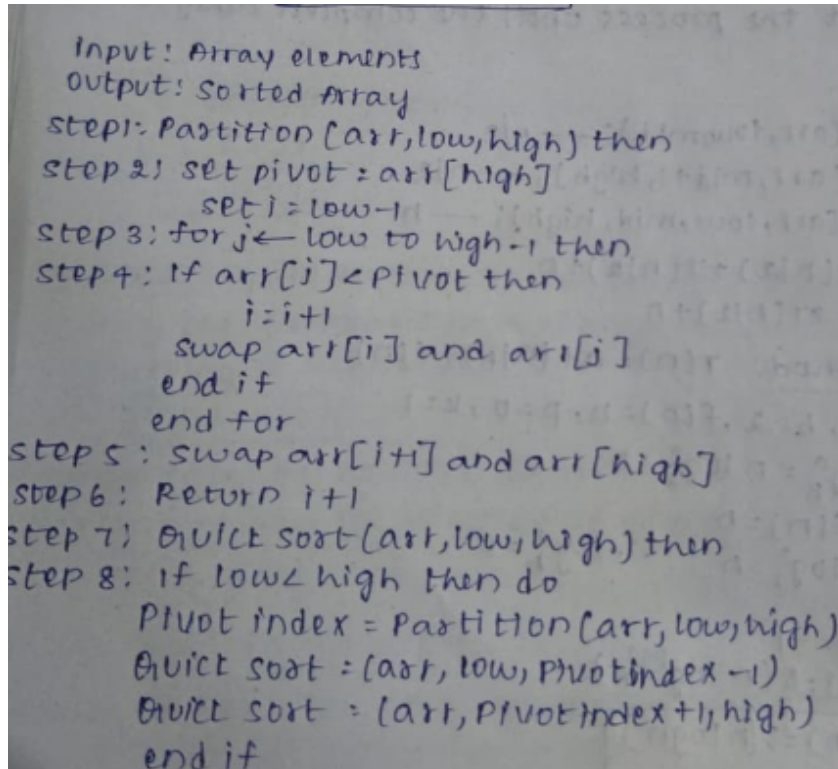
Best case: Eg: 1 2 3 4 5 6
 The array is sorted and merge sort still divides the array and merge the parts
 $\therefore T(n) = O(n \log n)$
Average case: Eg: 4 2 6 1 5 3
 The array is divided into smaller parts and the merged in sorted order.
 $\therefore T(n) = O(n \log n)$
Worst case: Eg: 6 5 4 3 2 1
 The array is in reverse order and merge sort still divides and merges all elements
 $\therefore T(n) = O(n \log n)$

Merge sort Conclusion :

Thus, the Merge Sort algorithm was successfully implemented and studied. It sorts the elements by dividing the array into smaller parts and then merging them in sorted order. It is efficient for large datasets and has a time complexity of **$O(n \log n)$** in the best, average, and worst cases.

5) Quick Sort:

Algorithm:



```
Input: Array elements
Output: Sorted Array
step 1: Partition (arr, low, high) then
step 2: set pivot = arr[high]
        set i = low - 1
step 3: for j ← low to high - 1 then
step 4: If arr[j] < Pivot then
        i = i + 1
        swap arr[i] and arr[j]
        end if
    end for
step 5: swap arr[i + 1] and arr[high]
step 6: Return i + 1
step 7: Quick sort (arr, low, high) then
step 8: if low < high then do
        Pivot index = Partition (arr, low, high)
        Quick sort = (arr, low, Pivot index - 1)
        Quick sort = (arr, Pivot index + 1, high)
    end if
```

Program :

```
#include <iostream>

using namespace std;

class QuickSort
{
public:
    int arr[100], n;

    void arrayInput()
    {
        cout << "Enter number of elements: ";

        cin >> n;
```



```
    cout << "Enter the elements: ";

    for (int i = 0; i < n; i++)

    {

        cin >> arr[i];

    }

}

void printArray()

{

    for (int i = 0; i < n; i++)

    {

        cout << arr[i] << " ";

    }

    cout << endl;

}

int partition(int arr[], int lb, int ub)

{

    int pivot = arr[lb];

    int start = lb;

    int end = ub;

    while (start < end)

    {

        while (arr[start] <= pivot && start < ub)

        {

            start++;

        }


```

```
while (arr[end] > pivot)
{
    end--;
}

if (start < end)
{
    swap(arr[start], arr[end]);
}
}

swap(arr[lb], arr[end]);

return end;
}

void quicksort(int arr[], int lb, int ub)
{
    if (lb < ub)
    {
        int location = partition(arr, lb, ub);

        quicksort(arr, lb, location - 1);

        quicksort(arr, location + 1, ub);
    }
}

void sort()
{
    quicksort(arr, 0, n - 1);
}
```

};

int main()

{

QuickSort qs;

qs.arrayInput();

cout << "\nBefore Sorting: ";

qs.printArray();

qs.sort();

cout << "After Sorting: ";

qs.printArray();

return 0;

}

Output:

```
PS C:\Users\vemal\OneDrive\Documents\DSA> g++ quick.cpp -o quick.exe
PS C:\Users\vemal\OneDrive\Documents\DSA> .\quick.exe
Enter number of elements: 5
Enter the elements: 12 14 34 78 56

Before Sorting: 12 14 34 78 56
After Sorting: 12 14 34 56 78
```

Time complexity :

Quick sort (arr, low, hi-1) : — $n/2$
 Quick sort (arr, pi+1, high) : — $n/2$
 Partition (arr, low, high) : — n
 $T(n) = T(n/2) + T(n/2) + n$
 $T(n) = 2T(n/2) + n$
 Master Method : $T(n) = 2T(n/2) + n$
 $a=2, b=2, f(n)=n, n=0, f=n$
 $n \log_b a = n \log_2 2 = n$
 $\therefore \log_b a = k$
 Case 2: $P > -1$
 $T(n) = O(n \log n^{P+1})$
 $T(n) = O(n \log n)$
 Best case: Eg: Arr: 4 2 6 1 5 3 7
 Pivot: 4
 The pivot divides the array into roughly equal parts
 $\therefore T(n) = O(n \log n)$
 Average case: Arr: 5 2 8 1 7 3 6
 Pivot: 5
 Left: 2, 3 Pivot: 5
 Right: 8 7 6
 $\therefore T(n) = O(n \log n)$
 Worst case: Arr: 1 2 3 4 5 6 7
 Pivot: 1
 [1] [2 3 4 5 6 7]
 [2] [3 4 5 6 7]
 $\therefore T(n) = O(n^2)$

Quicksort Conclusion :

Thus, the Quick Sort algorithm was successfully implemented and studied. It sorts the elements by selecting a pivot and partitioning the array around the pivot. It is generally efficient for large datasets, with an average-case time complexity of $O(n \log n)$. Its best-case complexity is $O(n \log n)$, while the worst-case complexity is $O(n^2)$.

Conclusion:

Thus, the sorting algorithms Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort were successfully implemented and their time complexities were analysed. The practical helped in understanding their performance and efficiency for different input sizes.